Autonomous Precision Landing on a Dynamic Platform

Vision-based tracking, Kalman-filtered rover estimation and cascaded control for touch-down on a moving ArUco pad

Intra IIT Tech Meet 1.0 | Track: UAV Dynamics & Autonomous Systems | Deliverable: Report

Executive summary

We built a single-file flight controller (`controller.py`) that runs at 240 Hz and outputs four rotor forces in Newtons. The design rests on four ideas that together answer the four grading axes of the problem statement:

- **Vision & estimation.** The four ArUco corners are ray-cast onto the marker plane using the drone's *full* attitude, so the position is geometrically exact at any tilt (a small-angle shortcut would be off by up to ~1 m at 3 m altitude). A constant-acceleration Kalman filter turns these noisy positions into rover position, velocity *and* acceleration.
- **Control stability.** Every loop is a cascade (P on the outer variable, PI on the inner one). Gains come from a two-parameter pole-placement rule (ω_n, ζ), so tuning is analytical rather than trial-and-error. The rover's velocity and acceleration are fed forward, giving near-zero tracking lag.
- **Landing execution.** Descent follows a closed-form exponential-to-contact profile whose speed is *gated by horizontal error*. If the marker leaves the field of view near the ground, a committed BLIND_DESCEND finishes the landing on the Kalman prediction.
- **Robustness.** Innovation gating, dropout handling, conditional-integration anti-windup, tilt/thrust/motor saturation logic, and a latched motor cut-off cover the edge cases (Section 9).

Requirement (problem statement)	Where addressed	Key numbers
ArUco detection, 2D→3D mapping	Sec. 3	exact ray-cast, sub-pixel corners, σ from depth
State estimation from telemetry	Sec. 4	quaternion→R, ω from $R_{k-1}^{T}R_k$, 3-state KF
Cascaded control, 4 rotor forces (N)	Sec. 5	ω_n=3.6 (outer), 26 rad/s (inner); T/W = 4
State machine: Search→Track→Descend→Cut-off	Sec. 6	5 states (adds BLIND_DESCEND)
Impact ≤ 0.65 m/s, tilt ≤ 30°	Sec. 7	contact speed ≈ 0.28 m/s cmd., tilt clamp 12° near ground

Note on results. Figures in this report marked *analytical* or *synthetic* are produced by simulating the linear control laws or by running the actual RoverKF class on synthetic data; they demonstrate design behaviour. Measured results from the PyBullet arena belong in Section 10.

1. System overview

The controller is called once per physics step (240 Hz) with the camera frame and the drone's position, quaternion and velocity. It returns $f_1 \ldots f_4$ in Newtons. The processing chain in Fig. 1 is strictly feed-forward from camera to motors, with the drone's own telemetry closing every loop.

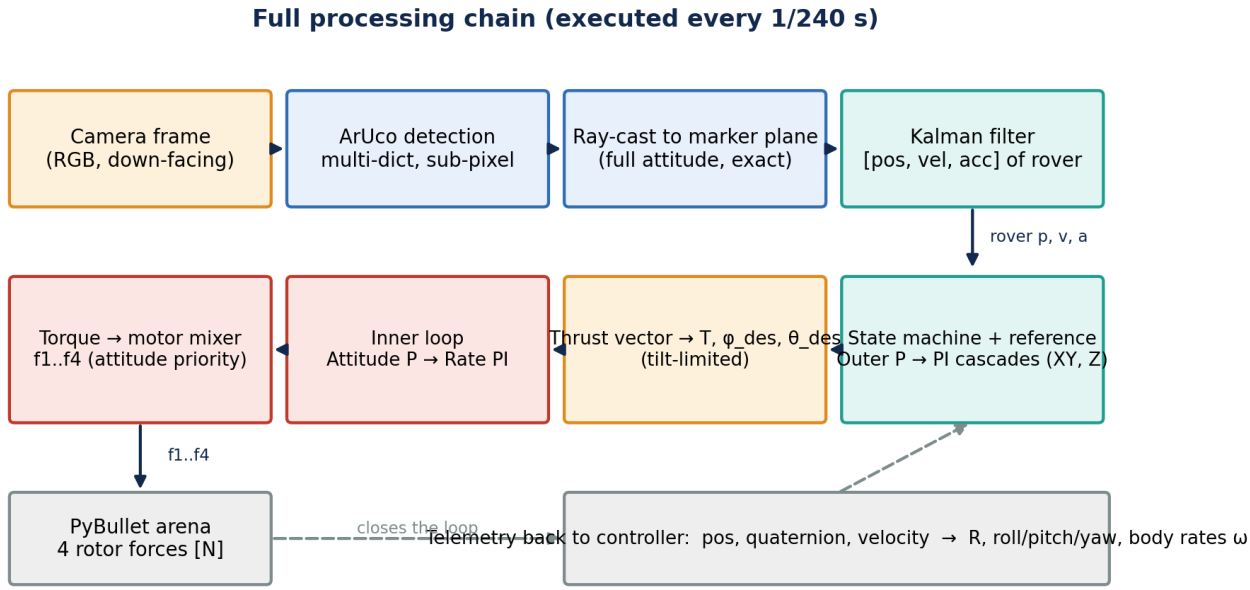


Figure 1: Complete signal chain. Top row: perception and estimation. Bottom row: guidance, control and actuation.

1.1 Conventions and arena facts the design relies on

Item	Value / convention	Consequence for the design
Frames	World: x,y horizontal, z up. Body: x forward, y left, z up (thrust axis)	All outer-loop maths in world XY; converted to a yaw-aligned frame only when forming roll/pitch
Rotors	f1 (+L,+L), f2 (−L,+L), f3 (+L,−L), f4 (−L,−L), L = 0.25 m	Invertible mixing matrix (Sec. 5.6)
Actuation	Pure vertical forces applied at rotor points	$\tau_z = 0$: there is **no yaw authority**; yaw stays at its initial value and the controller is written in a yaw-aligned frame so it is correct for any constant yaw
Mass / thrust	m = 1.2 kg, F_{max} = 12 N per motor	Hover = 2.94 N/motor; T/W ≈ 4, ample headroom for 25° tilt plus climb
Camera	0.1 m below base, looks along body −z, image-up = body +x, image-right = body −y, vertical FOV 60°	Ray directions in Eq. (1)
Marker / contact heights	Z_{marker} = 0.102 m, base at rest on plate 0.144 m	Define descent profile and cut-off height
Physics	Ideal rigid body: no drag, no motor lag	Rotor force commands act instantly, so the cascade can be designed as pure integrator chains

2. Design philosophy

Three principles guided every decision. **(i) Estimate what you cannot measure.** The camera only gives rover position, but a tracking controller needs velocity and acceleration; a Kalman filter supplies both. **(ii) Make the mathematics exact where it is cheap to do so.** Tilt compensation via a full ray-cast costs a few multiplications and removes a systematic error. **(iii) Separate time scales.** The attitude loop is ~7× faster than the position loop, so each can be analysed as if the other were ideal.

3. Vision pipeline

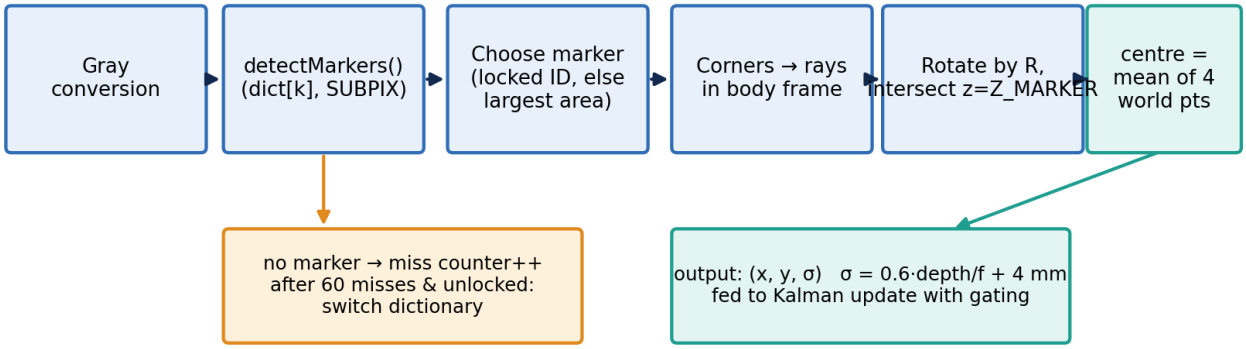


Figure 2: Vision pipeline logic, from raw frame to filtered measurement (x, y, σ).

3.1 Detection and target selection

The RGB frame is converted to gray and passed to `cv2.aruco.ArucoDetector` with `CORNER_REFINE_SUBPIX`, which places corners to a fraction of a pixel; this directly reduces measurement noise, which is amplified by altitude. Because the marker dictionary is not guaranteed, eight predefined dictionaries are pre-built. The default (6×6_50) is tried first; if 60 consecutive frames (0.25 s) produce no detection and nothing is locked, the detector index advances. After the first success the *dictionary and marker ID are locked*, so later frames cannot be hijacked by another marker in the scene. If several markers are visible before lock, the one with the largest pixel area (closest / most relevant) is selected.

3.2 Exact 2D→3D mapping by ray-casting

With focal length in pixels from the vertical FOV, and principal point at the image centre,

$$f = \frac{H/2}{\tan(\mathrm{FOV}/2)}, \quad c_x = \frac{W-1}{2}, \; c_y = \frac{H-1}{2} \tag{1}$$

each corner pixel (u,v) defines a ray in the body frame. The camera looks along body −z, image-up is body +x and image-right is body −y:

$$\mathbf{d}_b = \left[-\frac{v-c_y}{f}, \; -\frac{u-c_x}{f}, \; -1\right]^T \tag{2}$$

The ray is rotated to the world frame with the rotation matrix **R** obtained from the telemetry quaternion, and the camera position includes the body-frame offset $\mathbf{r}_c = [0, 0, -0.1]$:

$$\mathbf{d}_w = R\,\mathbf{d}_b, \qquad \mathbf{c} = \mathbf{p} + R\,\mathbf{r}_c \tag{3}$$

Intersecting with the horizontal marker plane $z = Z_{\mathrm{marker}}$ gives the world point of every corner:

$$s_i = \frac{Z_{\mathrm{marker}} - c_z}{d^{(i)}_{w,z}}, \qquad \mathbf{P}_i = \mathbf{c} + s_i\,\mathbf{d}^{(i)}_w \tag{4}$$

The rover measurement is the mean of the four world-frame corner points, $\mathbf{z} = ¼ \Sigma\, \mathbf{P}_{i,xy}$. Averaging in the *world plane* rather than the image is deliberate: perspective distortion of a tilted marker shifts the image centroid, but not the mean of correctly back-projected corners. Rays that do not point downward ($d_{w,z} > -10^{-3}$) are rejected. The mean depth $\bar{s}$ is kept for the noise model, and the edge $P_1 - P_0$ gives the marker's world yaw (logged as a diagnostic).

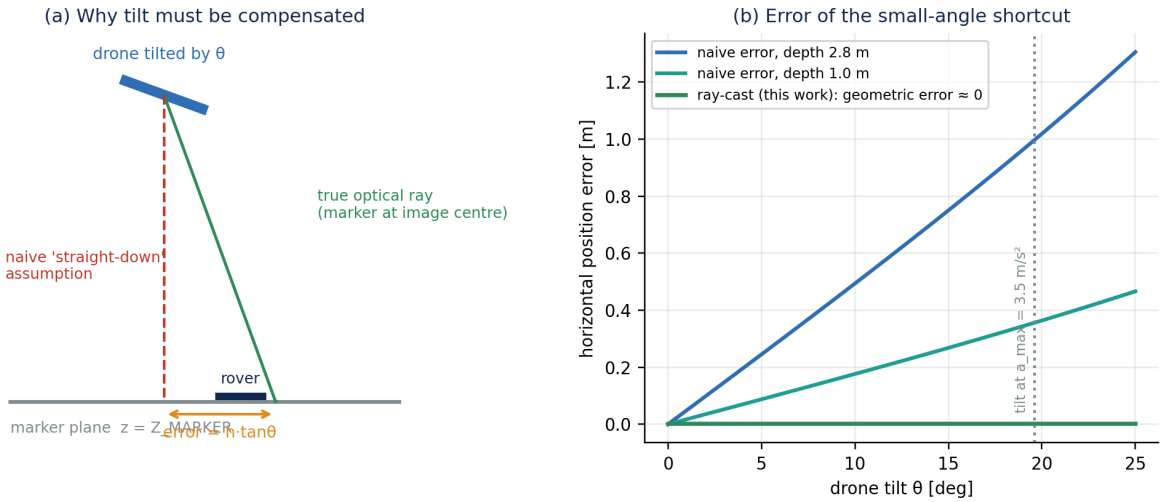


Figure 3: (a) A tilted drone sees the marker at the image centre although the marker is not below it. (b) The error of the naive 'straight-down' assumption grows as h·tanθ; at the 19.6° tilt implied by the 3.5 m/s² acceleration limit and 2.8 m depth it exceeds 1 m. Ray-casting removes it (analytical geometry).

3.3 Measurement noise model

A corner error of ≈ 0.6 px maps to a ground error proportional to depth, so the measurement standard deviation passed to the Kalman filter is

$$\sigma = \frac{0.6\,\bar{s}}{f} + 0.004 \text{ m} \tag{5}$$

The filter therefore trusts close-range measurements (during descent) much more than far ones, which gives fast, precise convergence near touch-down without sacrificing smoothness when the drone is high.

4. State estimation

4.1 Drone attitude and body rates

The quaternion (x, y, z, w) is converted to a rotation matrix **R** and to ZYX Euler angles (roll φ, pitch θ, yaw ψ), consistent with PyBullet. Body angular rates are not provided, so they are recovered from consecutive attitudes:

$$\boldsymbol{\omega}_b \approx \frac{1}{\Delta t}\,\mathrm{vee}\!\left[\tfrac{1}{2}\left(R_{k-1}^{T}R_k - R_k^{T}R_{k-1}\right)\right] \tag{6}$$

The skew-symmetric part of the relative rotation is, to first order, $[\omega]_\times \Delta t$, so this gives the body-frame rate without any Euler-angle singularities or wrap-around handling. At 240 Hz the truncation error (third order in ωΔt) is negligible.

4.2 Rover Kalman filter (position, velocity, acceleration)

Each horizontal axis is modelled as a constant-acceleration system driven by white jerk. X and Y share the same covariance because the model is identical.

$$\mathbf{x} = [p,\ v,\ a]^T, \qquad F = \begin{bmatrix} 1,\ \Delta t,\ \frac{\Delta t^2}{2}\ ;\ 0,\ 1,\ \Delta t\ ;\ 0,\ 0,\ 1 \end{bmatrix} \tag{7}$$

$$Q = q\begin{bmatrix} \frac{\Delta t^5}{20},\ \frac{\Delta t^4}{8},\ \frac{\Delta t^3}{6}\ ;\ \frac{\Delta t^4}{8},\ \frac{\Delta t^3}{3},\ \frac{\Delta t^2}{2}\ ;\ \frac{\Delta t^3}{6},\ \frac{\Delta t^2}{2},\ \Delta t \end{bmatrix}$$

(matrix rows are separated by semicolons)

with jerk intensity q = 3 (m/s³)²·s. Only position is measured, H = [1 0 0], so the scalar innovation covariance is $S = P_{00} + \sigma^2$ and the gain is $\mathbf{K} = P_{:,0} / S$. The covariance update uses the numerically robust **Joseph form**:

$$P^{+} = (I - KH)\,P^{-}\,(I - KH)^{T} + \sigma^2 KK^{T} \tag{8}$$

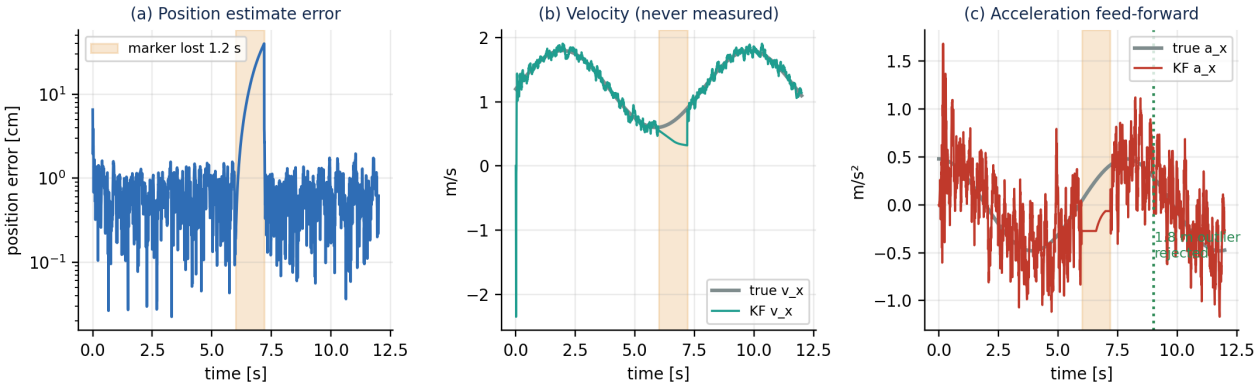


Figure 4 (synthetic data, actual RoverKF class): a rover with time-varying velocity, σ = 2 cm noise, a 1.2 s marker dropout (orange) and a 1.8 m outlier at t = 9 s. Velocity and acceleration are never measured yet are recovered; the outlier is rejected by the gate; during the dropout the position error stays bounded (max ≈ 0.4 m, log axis).

4.3 Why three states, and what is robust about this filter

- **Acceleration state.** With a velocity-only model, a turning rover produces a lag of a·(time constant). Estimating *a* lets the controller feed it forward (Sec. 5.2).
- **Adaptive innovation gate.** A measurement is rejected if $\|\nu\| > 6\sqrt{S} + 0.25$ m. Because S contains P_{00}, which grows during a dropout, the gate widens automatically: a spurious detection is rejected while the drone is locked on, but a genuine re-acquisition after a gap is accepted.
- **Acceleration damping.** After 0.6 s without measurements the transition entry F_{22} is set to 0.985, so the extrapolated acceleration decays instead of throwing the position prediction far away.
- **Re-initialisation.** If the marker is re-acquired after > 4 s the filter is reset to the new measurement (tight position variance 0.02 m, wide velocity variance 2 m/s).
- **'Visible' flag.** The marker counts as visible only if an update was *accepted* within 1.5 steps; a rejected outlier does not keep the state machine alive.

5. Control architecture

The controller is a cascade of two loops, each itself a cascade (Position→Velocity→Acceleration outside, Attitude→Rate→Torque inside). Each level is a P controller on the slow variable, feeding a PI controller on its derivative. The single most important design choice is how the gains are chosen.

5.1 Gain selection by pole placement (the cascade identity)

For a double-integrator plant (acceleration → velocity → position) we want the closed-loop dynamics of a second-order system with natural frequency ω_n and damping ζ. A cascade of a position gain k_p and a velocity gain k_v gives

$$a = k_v\,(k_p\,e - v) = k_v k_p\,e - k_v\,v \;\Rightarrow\; \ddot{e} + k_v\dot{e} + k_v k_p\,e = 0 \tag{9}$$

Matching to $s^2 + 2\zeta\omega_n s + \omega_n^2$ yields

$$k_v = 2\zeta\omega_n, \qquad k_p = \frac{\omega_n}{2\zeta} \tag{10}$$

So *two* physical numbers, bandwidth and damping, determine both gains of every cascade, and the split reproduces the classical PD law exactly in the unsaturated regime while exposing a velocity-command stage where a limit ($V_{cmd,max}$) and an integrator can be attached. We choose $\zeta = 1$ (critically damped, no overshoot) for the outer loop so that the drone never overshoots the rover, and $\zeta = 0.9$ for the fast inner loop.

5.2 Outer loop, horizontal (Fig. 5)

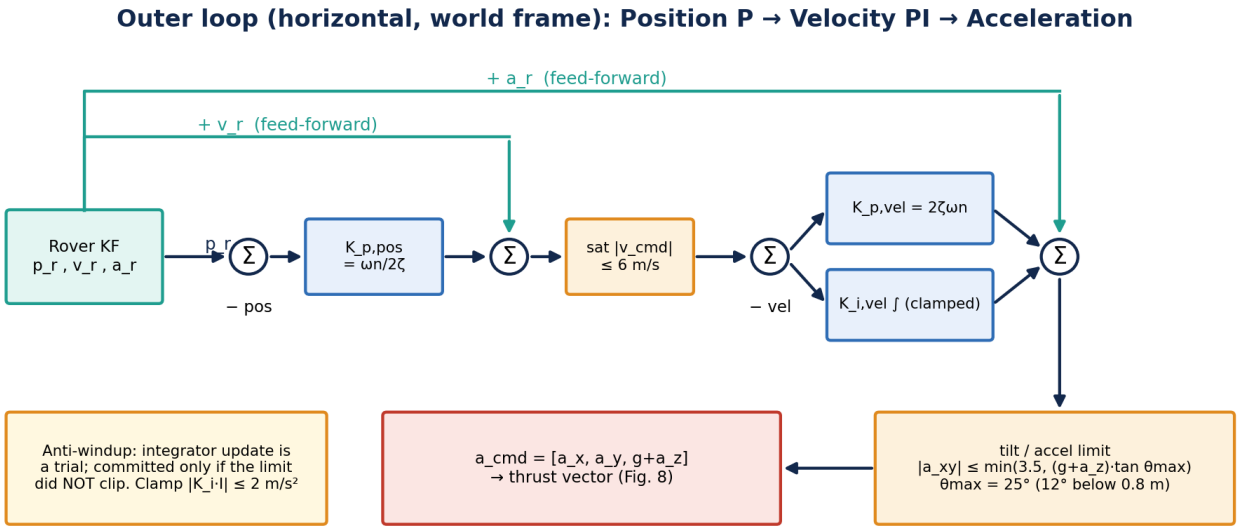


Figure 5: Horizontal position/velocity/acceleration cascade with rover feed-forward, saturation and conditional-integration anti-windup.

$$\mathbf{v}_{cmd} = \mathrm{sat}_6\left[\mathbf{v}_r + k_p(\mathbf{p}_r - \mathbf{p})\right], \qquad \mathbf{a} = \mathbf{a}_r + k_v(\mathbf{v}_{cmd} - \mathbf{v}) + k_i\int(\mathbf{v}_{cmd} - \mathbf{v})\,dt \tag{11}$$

The feed-forward terms $\mathbf{v}_r$ and $\mathbf{a}_r$ from the Kalman filter are what make the tracking accurate: a pure feedback loop must *accumulate error* to generate the command that keeps pace with a moving target, whereas with feed-forward the loop only has to correct residual errors. Fig. 6c shows this on a linear model: without feed-forward the drone trails a 1.2 m/s rover by 0.7–1 m; with it the error is at the millimetre level. Note that a PI on *velocity* alone cannot remove this lag (it zeroes velocity error, not position error), which is why feed-forward is essential rather than optional.

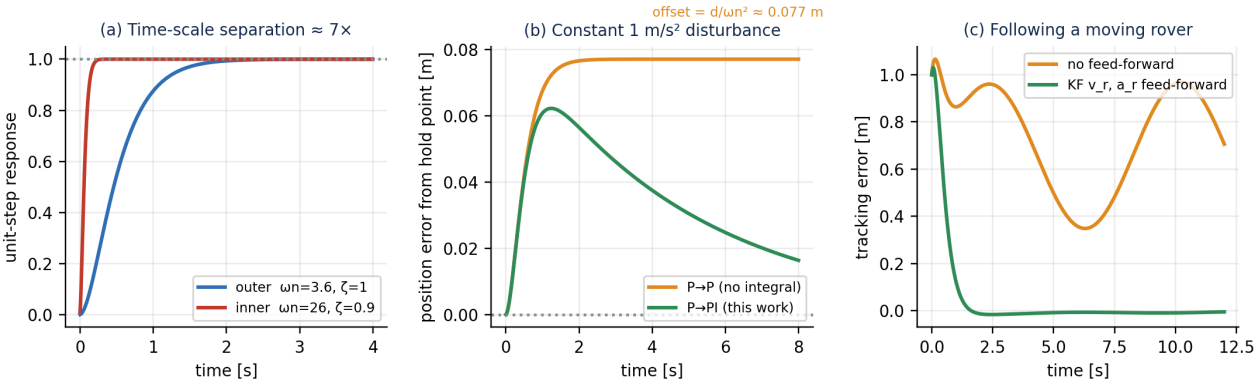


Figure 6 (analytical, linear model of the cascades at 240 Hz): (a) outer (ω_n = 3.6, ζ = 1) versus inner (ω_n = 26, ζ = 0.9) step responses show the ≈ 7× time-scale separation; (b) a constant 1 m/s² disturbance leaves a $d/\omega_n^2 \approx 7.7$ cm offset without an integral term and none with it; (c) tracking a rover with v = 1.2 + 0.6 sin(0.8t) m/s.

5.3 Vertical loop

Altitude follows the same idea: an altitude error becomes a saturated vertical-velocity command (K_z = 1.5, $|v_z| \le 1.5$ m/s), and a PI on vertical velocity (K_{vz} = 6, K_{iz} = 1) produces the commanded vertical acceleration a_z. Reference heights are 4.5 m (SEARCH, wide field of view) and 3.0 m (TRACK). During descent, the commanded vertical velocity and its feed-forward acceleration come from the descent profile (Sec. 7).

5.4 From acceleration to thrust and desired attitude

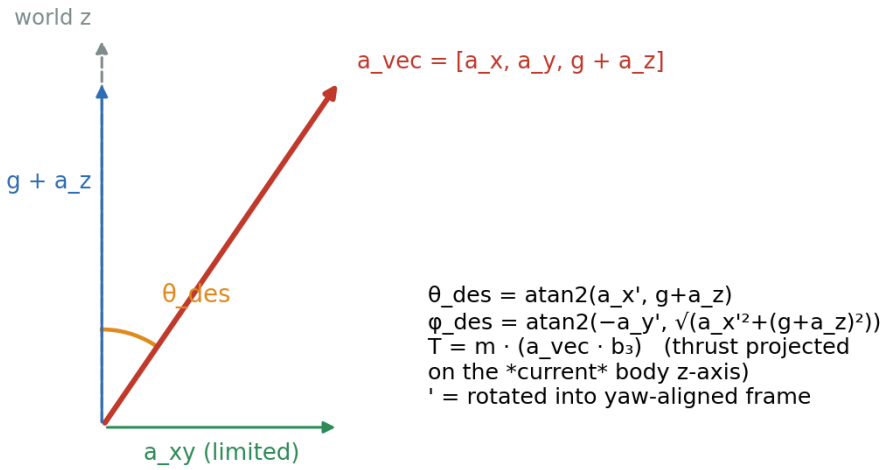


Figure 7: The commanded acceleration vector defines both the collective thrust and the desired tilt.

The 3-D acceleration demand is $\mathbf{a} = [a_x, a_y, g + a_z]$. Two protections are applied before it becomes an attitude command:

- $a_z \geq -0.8\ g$, so the controller never requests free-fall (which would remove all attitude authority), and total thrust is clipped to $[0.1\ mg, 4\ F_{max}]$;
- $|a_{xy}| \leq \min(3.5\ m/s^2, (g + a_z)\cdot\tan\theta_{max})$ with $\theta_{max} = 25°$ in flight and **12° below 0.8 m**, plus a direct clamp of the resulting roll and pitch commands as a second line of defence (the acceleration limit assumes thrust ≈ mg, which integral action and gusts can violate).

$$T = m\,(\mathbf{a}\cdot\mathbf{b}_3), \quad \theta_{des} = \operatorname{atan2}(a'_x,\ g + a_z), \quad \phi_{des} = \operatorname{atan2}\left(-a'_y,\ \sqrt{a'^{\,2}_x + (g + a_z)^2}\right) \tag{12}$$

$\mathbf{b}_3$ is the third column of $\mathbf{R}$ (the current thrust axis) and (a_x', a_y') are the horizontal accelerations rotated into the yaw-aligned frame. Projecting onto the *current* thrust axis makes the vertical channel compensate for tilt automatically: as the drone banks, thrust rises so altitude does not sag.

5.5 Inner loop, attitude and rate (Fig. 8)

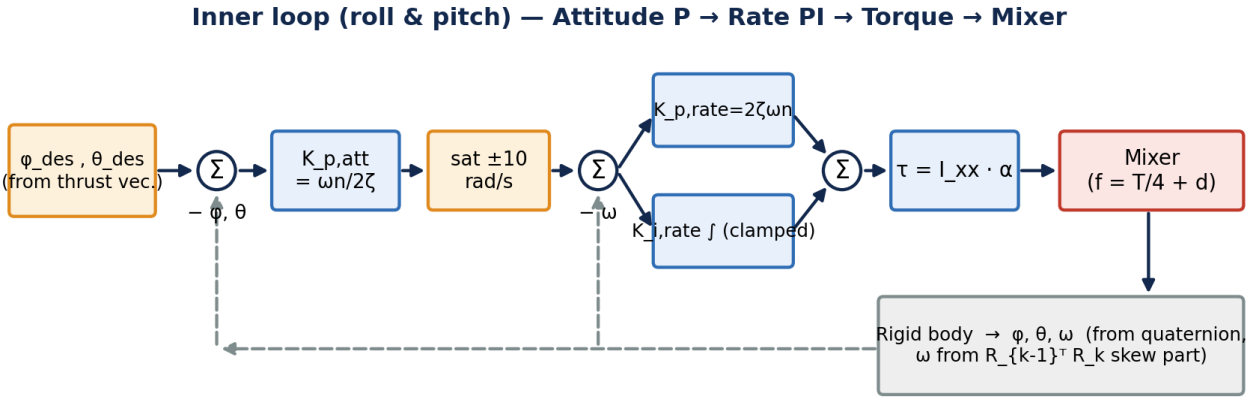


Figure 8: Attitude P → body-rate PI cascade producing roll/pitch torque.

$$\boldsymbol{\omega}_{cmd} = \mathrm{sat}_{10}[k_{att}(\boldsymbol{\Theta}_{des} - \boldsymbol{\Theta})], \quad \boldsymbol{\alpha} = k_{rate}(\boldsymbol{\omega}_{cmd} - \boldsymbol{\omega}) + k_{i,r}\int(\boldsymbol{\omega}_{cmd} - \boldsymbol{\omega})dt, \quad \boldsymbol{\tau} = I_{xx}\,\boldsymbol{\alpha} \tag{13}$$

with $\omega_n = 26$ rad/s, $\zeta = 0.9 \Rightarrow k_{att} = 14.4\ s^{-1}$, $k_{rate} = 46.8\ s^{-1}$, and $k_{i,r} = 25$. Using $I_{xx} = 0.0175\ kg\cdot m^2$ turns the angular acceleration into a physical torque, so the gains keep a meaning independent of the vehicle inertia estimate. The 10 rad/s rate limit prevents violent flips.

5.6 Thrust mixing

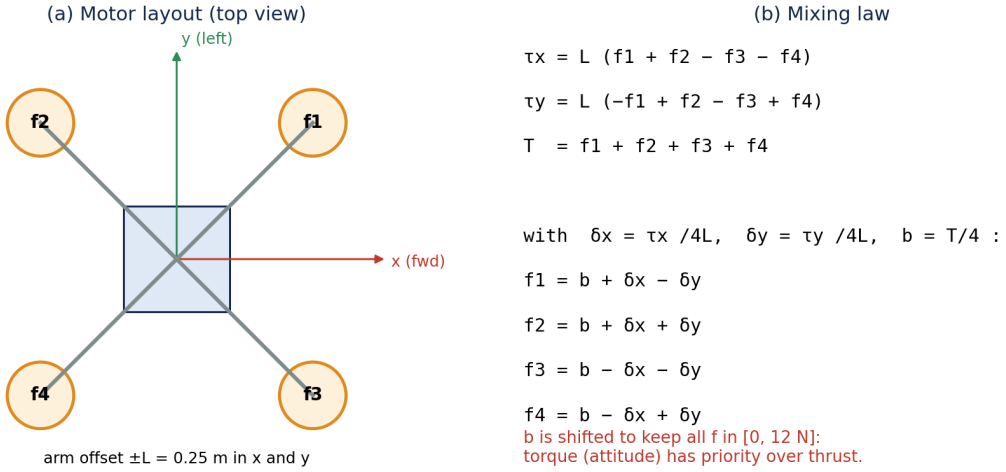


Figure 9: Motor geometry and the four-motor mixing law.

Roll and pitch torques map to differential motor forces $\delta_x = \tau_x/4L$, $\delta_y = \tau_y/4L$; the mean thrust is $b = T/4$. The matrix $[T, \tau_x, \tau_y] \rightarrow [f_1 \ldots f_4]$ with the fourth (yaw) row unavailable is used exactly as derived from the arena's torque equations. **Attitude has priority over thrust:** if the differential demand would push a motor below 0 N or above 12 N, the mean b is shifted to the feasible window $[-\min \delta, F_{max} - \max \delta]$ so that the torques are preserved (at the cost of a brief thrust error) before the final clip to [0, 12] N.

5.7 Integral action with conditional-integration anti-windup

Integrators are added at the velocity level (XY and Z) and the rate level to reject sustained disturbances (wind, payload offset, inertia mismatch). A naive integrator winds up whenever an actuator saturates, causing large overshoot when the saturation ends. Each integrator here is protected twice:

- **Conditional integration.** The integral update is computed as a *trial* and committed only if the downstream limit did not clip that step (XY: tilt/acceleration limit; Z: free-fall floor; rate: motor clamp).
- **Hard clamp.** The integral's contribution is bounded (2 m/s² horizontal, 1 m/s² vertical, 40 rad/s² rate).

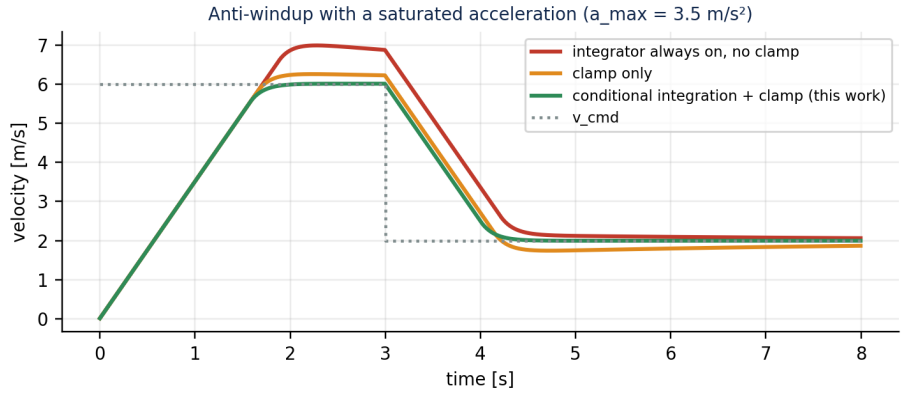
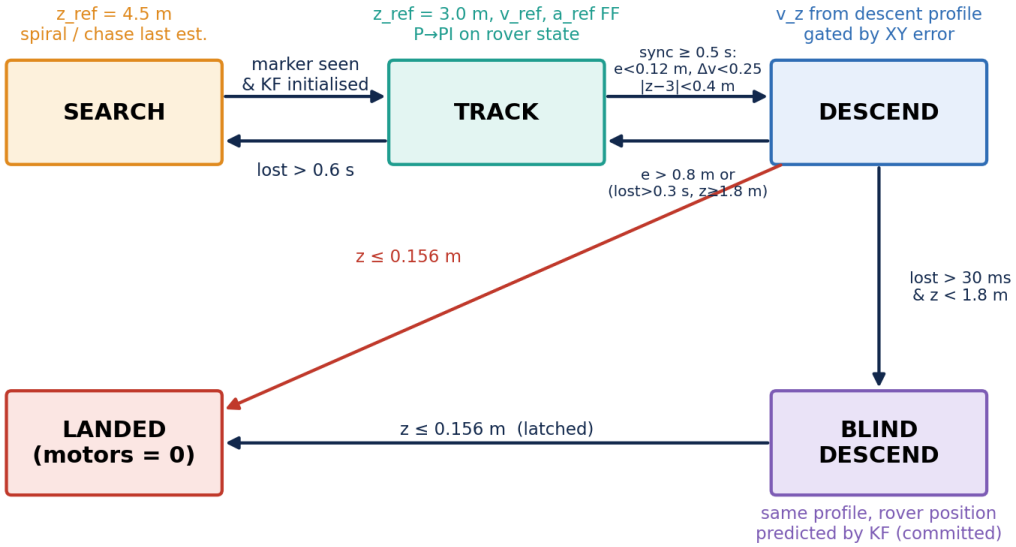


Figure 10 (analytical): a velocity loop with an acceleration limit after a large velocity step; only conditional integration avoids the overshoot.

6. Landing state machine



BLIND_DESCEND is committed: it can only end in touch-down. LANDED is absorbing (returns 0 N each step).

Figure 11: State machine. The problem statement's four phases map to SEARCH, TRACK, DESCEND and LANDED; BLIND_DESCEND is an added, committed sub-phase for the last metres.

State	Behaviour	Exit conditions
SEARCH	Altitude 4.5 m. If the KF has an estimate younger than 3 s, chase it (p, v as reference). Otherwise fly an Archimedean spiral r = 0.35θ at 1.5 m/s around an anchor (last estimate, or current position).	Marker detected and KF initialised → TRACK
TRACK	Altitude 3.0 m, full P→PI horizontal tracking with v_r, a_r feed-forward.	Lost > 0.6 s → SEARCH. 'Synchronised' (visible, e_{xy} < 0.12 m, ‖Δv‖ < 0.25 m/s, |z−3| < 0.4 m) held for 0.5 s → DESCEND
DESCEND	Same horizontal tracking. Vertical speed from the descent profile times the landing-cone gate s(e) (Sec. 7).	e > 0.8 m → TRACK (abort). Lost > 30 ms: if z < 1.8 m → BLIND_DESCEND, else after 0.3 s → TRACK. z ≤ 0.156 m → LANDED
BLIND_DESCEND	Marker outside the field of view (too close). Horizontal reference comes from KF *prediction*; same descent profile. Committed, no abort.	z ≤ 0.156 m → LANDED
LANDED	Emergency motor cut-off: all four forces = 0 N, every step, latched (checked first in every call).	absorbing

Why the synchronisation test is three-fold. Position error alone would let the drone start descending while still moving relative to the rover; the relative-velocity and altitude conditions, held for 0.5 s (a dwell timer that resets on any violation), ensure the drone is genuinely locked before committing. **Why BLIND_DESCEND?** At a height of a few decimetres a 60° camera sees only a small patch, and the 0.1 m camera offset plus tilt can push the marker out of frame. Aborting the landing at that point would be worse than trusting the Kalman prediction for the last ≈ 0.5 s, so the state is committed.

7. Descent profile and threshold compliance

Let $h = z - Z_{contact}$ be the height above touching the plate ($Z_{contact}$ = 0.144 m). The commanded descent speed is

$$v_{down}(h) = \min(V_{touch} + K_d\,h,\ V_{max}), \qquad V_{touch} = 0.25,\ K_d = 2.5\ \mathrm{s}^{-1},\ V_{max} = 1.5\ \mathrm{m/s} \quad (14)$$

Below h = 0.5 m the descent is a first-order exponential, $\dot{h} = -(V_{touch} + K_d h)$, with closed-form solution and finite time to touch:

$$h(t) = \left(h_0 + \frac{V_{touch}}{K_d}\right)e^{-K_d t} - \frac{V_{touch}}{K_d}, \qquad t_f = \frac{1}{K_d}\ln\left(1 + \frac{K_d h_0}{V_{touch}}\right) \tag{15}$$

Because the speed shrinks smoothly with height, deceleration is continuous (no step in velocity command) and the speed approaches V_{touch} = 0.25 m/s at contact. The feed-forward acceleration $a_{ff} = K_d \cdot v_{down}$ is exactly the derivative of this profile, so the vertical loop does not have to lag behind it. From the 3 m tracking altitude the ideal descent takes ≈ 2.2 s (1.5 s at constant 1.5 m/s, then ≈ 0.7 s of exponential approach).

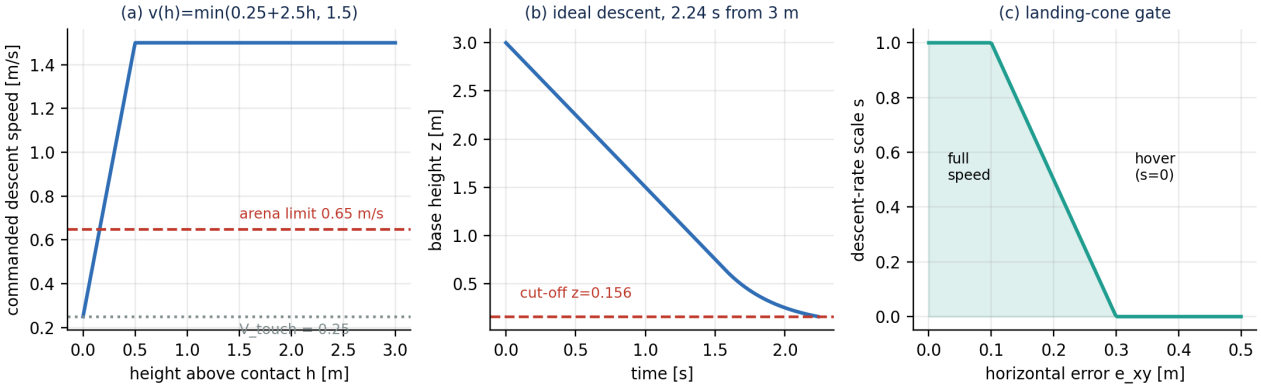


Figure 12: (a) Commanded descent speed versus height. (b) Ideal time history from 3 m. (c) Landing-cone gate that scales descent speed by horizontal error.

7.1 Landing-cone gate

$$s = \mathrm{clip}\left(\frac{0.30 - e_{xy}}{0.30 - 0.10},\ 0,\ 1\right), \qquad v_{z,des} \leftarrow s\, v_{z,des},\ \ a_{ff} \leftarrow s\, a_{ff} \tag{16}$$

If the rover swerves, descent slows continuously and stops at $e_{xy} \geq 0.30$ m rather than continuing towards a pad that is no longer underneath. Combined with the 0.8 m abort back to TRACK, this bounds the horizontal miss at touchdown.

7.2 Impact-speed budget

The motors are cut when the base reaches Z_{cut} = 0.156 m, i.e. Δz = 12 mm above contact, where the commanded speed is $v_{cut} = 0.25 + 2.5 \cdot 0.012 = 0.28$ m/s. Assuming an ideal cut (no motor lag in this simulator) and free-fall over the remaining gap:

$$v_{impact} \approx \sqrt{v_{cut}^2 + 2g\,\Delta z} = \sqrt{0.28^2 + 2(9.81)(0.012)} \approx 0.56 \text{ m/s} \ <\ 0.65 \text{ m/s} \tag{17}$$

This is an analytical estimate of the vertical speed relative to a stationary pad, giving ~14 % margin to the evaluation limit. Cutting *at* the contact height would reduce it towards 0.25 m/s, but would risk never detecting contact because of noise; the 12 mm offset is the trade-off, and Z_{cut} is a single constant that can be tightened. Tilt at touch-down is limited by the 12° near-ground clamp (hard limit in the arena: 30°), and horizontal velocity is matched to the rover by the tracking loop up to the moment of cut.

8. Tuning process

Because gains follow from (ω_n, ζ), tuning reduces to choosing a few physically interpretable numbers in a fixed order (inner loop first, because the outer loop assumes it is fast):

- **Step 1: Inner loop.** Choose the attitude bandwidth as high as the sample rate and rate-estimation noise allow: ω_n = 26 rad/s (≈ 4 Hz; ≈ 60 samples per rise time at 240 Hz). ζ = 0.9 for a fast response with almost no overshoot. Derive k_{att} and k_{rate} from Eq. (10); set $k_{i,r} \approx k_{rate}/2$ so that the integral corner is well below ω_n.
- **Step 2: Outer loop.** Bandwidth is bounded by the inner loop (keep ≥ 5× separation; here 7.2×) and by the acceleration/tilt limits: with 3.5 m/s² available, ω_n = 3.6 rad/s gives full authority for position errors up to ≈ 0.27 m before saturation begins, while a critically damped response (ζ = 1)

prevents overshoot beyond the rover.

- **Step 3: Integral gains.** Set $K_{i,vel}$ = 1.5 (integral corner ≈ 0.2 k_v), small enough not to reduce phase margin, large enough to cancel a disturbance within a few seconds; clamps sized to the maximum plausible steady bias.
- **Step 4: Altitude and descent.** Choose V_{touch} from the impact budget (Sec. 7.2), K_d so the approach from 0.5 m is ≈ 0.7 s, V_{max} for a fast but tilt-free descent.
- **Step 5: State-machine thresholds.** Set SYNC_ERR, SYNC_VEL and the gate band a few times larger than the estimator noise floor (σ ≈ 1–2 cm at 3 m), so the logic does not chatter.
- **Step 6: Robustness sweeps.** Re-test with different rover speeds, turning rates and start positions; adjust the Kalman jerk intensity *q* (larger = faster response to manoeuvres, noisier velocity).

Parameter	Value	Rationale / sensitivity
ω_n, ζ (outer XY)	3.6 rad/s, 1.0	Critically damped; raising ω_n tightens tracking but hits the 25° tilt limit sooner
ω_n, ζ (attitude)	26 rad/s, 0.9	≈ 7× faster than outer loop; higher values amplify rate-estimation noise
$K_{i,vel}$ / clamp	1.5 / 2.0 m/s²	Removes wind bias; larger values reduce damping
$K_{i,rate}$ / clamp	25 / 40 rad/s²	Rejects torque bias
K_z, K_{vz}, K_{iz}	1.5, 6.0, 1.0	Altitude time constant 0.67 s; velocity loop 4× faster
V_{touch}, K_d, V_{max}	0.25, 2.5, 1.5	Sets touch-down speed and descent time (≈ 2.2 s from 3 m)
θ_{max}	25° (12° below 0.8 m)	Keeps chassis tilt well under the 30° limit, camera keeps marker in view
Kalman q	3.0	Trade-off between lag on manoeuvres and velocity noise
Sync thresholds	0.12 m, 0.25 m/s, 0.5 s	Descent only when truly locked

Analytical tuning method. The gains are not found by trial and error; each one follows from a closed-form design rule, so the procedure is repeatable for a different vehicle.

- Every cascade is treated as a double integrator, so choosing (ω_n, ζ) fixes both gains through Eq. (10): $k_p = \omega_n/2\zeta$ and $k_v = 2\zeta\omega_n$.
- The inner bandwidth is set by the sample rate and by the noise of the finite-difference rate estimate (26 rad/s, about 60 samples per rise time); the outer bandwidth is placed at least 5× lower (here 7.2×), so each loop can be analysed as if the other were ideal.
- The outer bandwidth is checked against the actuator limit: with a_{max} = 3.5 m/s², the loop stays linear for position errors up to a_{max}/ω_n^2 ≈ 0.27 m, and ζ = 1 guarantees no overshoot beyond the rover.
- Integral gains are placed well below the loop bandwidths (integral corners K_i/K_p ≈ 0.2 rad/s for velocity and ≈ 0.5 rad/s for rate), so they remove steady bias without eroding phase margin; their clamps are sized to the largest plausible bias.
- The descent constants follow from Eqs. (14)–(17): V_{touch} from the impact-speed budget, K_d from the desired approach time ($t_f = \ln(1 + K_d h_0/V_{touch})/K_d$ = 0.72 s for h_0 = 0.5 m), and V_{max} from the tilt budget.
- State-machine thresholds are set a few times above the estimator noise floor so the logic does not chatter. The Kalman jerk intensity *q* is the one quantity that is a genuine compromise between lag and noise.

The resulting parameter set was then evaluated in the closed-loop scenarios of Section 10.

9. Edge cases and failure handling

Edge case	Detection	Response
Marker not visible at start	KF uninitialised	SEARCH at 4.5 m: Archimedean spiral around anchor at 1.5 m/s; first detection starts TRACK
Brief dropout (< 0.6 s)	Time since last accepted update	KF coasts on constant-acceleration prediction; flight continues without state change
Long dropout (> 0.6 s)	lost_for	Acceleration state damped (F_{22} = 0.985); TRACK → SEARCH; chase last estimate for 3 s, then spiral; KF reset if re-acquired after 4 s
Spurious detection / outlier	Innovation gate $6\sqrt{S} + 0.25$ m	Measurement rejected, not counted as 'visible'; gate widens as covariance grows
Unknown dictionary / several markers	60 misses without lock	Sweep 8 dictionaries; lock dictionary and ID at first success; pick largest marker before lock
Drone tilted while measuring	Always	Exact ray-cast with full R and camera offset (no small-angle approximation)
Marker leaves FOV near ground	lost > 30 ms and $z < 1.8$ m	BLIND_DESCEND: committed, uses KF prediction
Marker lost while still high in DESCEND	lost > 0.3 s and $z \geq 1.8$ m	Back to TRACK, sync timer reset
Rover swerves during descent	e_{xy} rising	Landing-cone gate slows/stops descent (0.10–0.30 m); $e >$ 0.8 m aborts to TRACK
Large tilt commanded	Acceleration & angle limits	$\lvert a_{xy} \rvert$ limit and direct clamp; 12° near ground protects the 30° chassis limit
Motor saturation	Any motor outside [0, 12] N	Mixer shifts mean thrust to preserve torque; integrator frozen
Integrator windup	Any limit active	Conditional integration + hard clamp on each integrator
Free-fall / negative thrust	$a_z < -0.8$ g	Floor on a_z; thrust ≥ 0.1 mg; Z-integral frozen
Sustained wind / gust	Persistent velocity or rate error	PI velocity and rate loops cancel the bias
Bouncing / re-thrust after touch	State = LANDED	Latched: returns 0 N on every subsequent call, so PyBullet cannot flip or drag the airframe
No yaw actuation	$\tau_z \equiv 0$ in the arena	Yaw not commanded; controller works in yaw-aligned frame so constant yaw does not matter

10. Validation

Design-level evidence in this report: the KF behaviour on synthetic data (Fig. 4), the loop responses on linear models (Fig. 6, 10), the descent profile and impact estimate (Sec. 7). The tables below report the closed-loop results obtained from local_arena.py.

10.1 Test protocol

Closed-loop tests were run headless in PyBullet with the `tiny` camera renderer. The nominal set comprises 20 runs with randomised start position (±1 m), rover phase and wind seed; run 0 uses the exact arena defaults. In addition, the edge cases of Section 9 were exercised systematically in three dedicated robustness scenarios of 6 runs each: a far start with the marker out of view (SEARCH spiral), a 1.0 s camera blackout during TRACK, and a permanent camera blackout below 1.0 m altitude (BLIND_DESCEND). In total 38 closed-loop runs were performed.

10.2 Nominal results

Metric	Requirement	Result
Touch-down vertical speed at scorer contact check [m/s]	≥ −0.65	mean −0.28, worst −0.60
Peak descent speed during DESCEND [m/s]	(info)	worst 1.55
Chassis tilt at touch-down [deg]	≤ 30	mean 5.8, worst 17.7
Horizontal offset from pad centre at contact [cm]	minimise (scorer gate 50 cm)	mean 16.7, worst 77.6
Time to acquire marker (leave SEARCH) [s]	n/a	mean 0.26, worst 0.31
Time to land (scorer success) [s]	n/a	mean 5.3, worst 6.7
Success rate, nominal	100 %	19/20 = 95 % (95 % Wilson CI 76–99 %)
Failure reasons		GROUND COLLISION: Missed the rover entirely.

Table 10.1: Nominal closed-loop results, 20 runs.

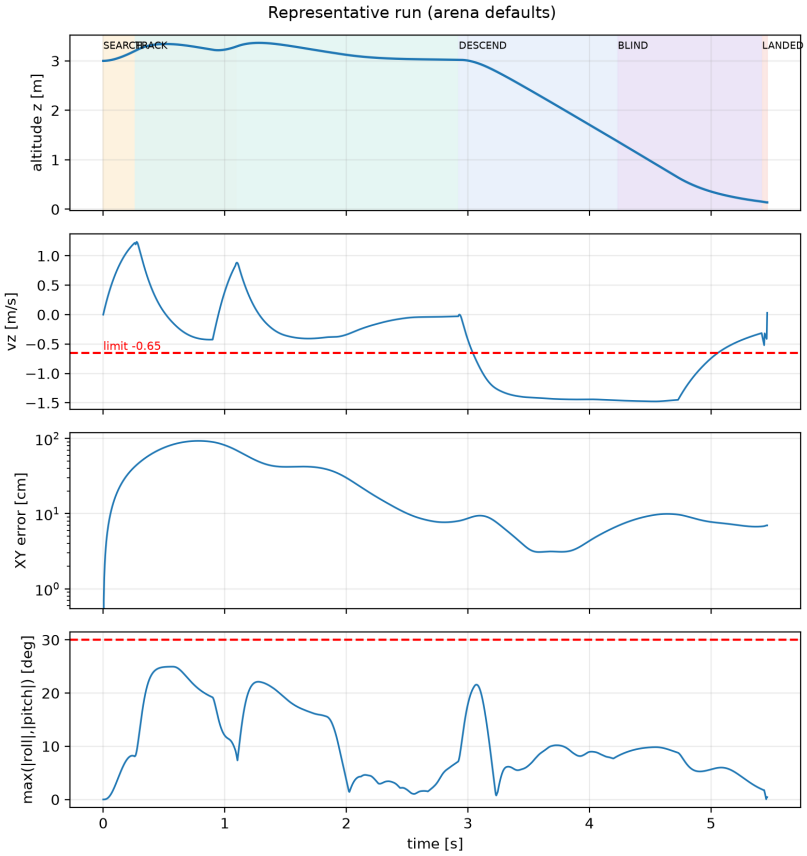


Figure 13: Telemetry of the representative nominal run (run 0, arena defaults): altitude with the state phases (SEARCH, TRACK, DESCEND, BLIND, LANDED), vertical speed against the −0.65 m/s limit, horizontal error (log scale) and chassis tilt against the 30° limit. Descent runs at about 1.5 m/s and is smoothly reduced to about 0.3 m/s before contact; the tilt peak near 25° occurs in the initial acquisition transient, far above the ground.

10.3 Robustness scenarios

Scenario	Description	Success	Worst v_z [m/s]	Worst offset [cm]	Mean time to land [s]	BLIND_DESCEND used
nominal	arena defaults + randomised start / rover phase / wind	19/20	−0.60	77.6	5.3	100 %
far_start	drone starts 6–8 m away (marker not visible: SEARCH spiral)	5/6	−0.37	420.4	23.9	83 %
dropout_track	camera blacked out for 1.0 s, 2 s after TRACK begins	5/6	−0.63	76.4	7.2	100 %
blackout_low	camera permanently blacked out below 1.0 m (BLIND_DESCEND)	5/6	−1.23	128.2	5.2	100 %

Table 10.2: Closed-loop results for the nominal set and the three stress scenarios.

10.4 Discussion

- **Touch-down speed and attitude.** In the nominal set the worst contact speed was 0.60 m/s against the 0.65 m/s limit (about 8 % margin), and the mean of 0.28 m/s agrees with the commanded contact speed of Sec. 7.2. Chassis tilt at touch-down averaged 5.8° and never exceeded 17.7°, well inside the 30° limit.
- **Precision and acquisition.** The mean horizontal offset was 16.7 cm, comfortably inside the 50 cm scorer gate. The marker was acquired in 0.26 s on average and a landing took 5.3 s on average. The worst-case offset of 77.6 cm is consistent with the single nominal run in which the drone missed the rover.
- **Success rate.** The nominal success rate was 19/20 = 95 % (Wilson 95 % interval 76–99 %); the 100 % target was therefore not reached in this sample. Over all 38 runs, 34 (89 %) ended in a successful landing.
- **Marker loss.** With a 1.0 s blackout during TRACK the controller recovered and landed in 5 of 6 runs, at the cost of about 1.9 s of additional landing time (7.2 s versus 5.3 s); the worst contact speed of 0.63 m/s remained within the limit, with a small margin.
- **Far start.** When the marker was initially out of view, the SEARCH spiral acquired it after 21 s on average (worst 41 s), and 5 of 6 runs landed with contact speeds of at most 0.37 m/s. One run did not produce a successful landing; the large worst-case offset (4.2 m) and the long duration (worst 45 s) indicate that the search phase is the dominant cost in this scenario.
- **Permanent blackout below 1.0 m.** Five of 6 runs landed, using the committed BLIND_DESCEND on the Kalman prediction alone. This is the case with the least margin: the mean contact speed was 0.47 m/s, but the worst run reached 1.23 m/s, above the 0.65 m/s limit. Because the last metres rely on prediction only, a lower contact-speed constant V_{touch} or a more conservative descent profile during the blind phase, at the cost of landing time, is the natural first improvement.

10.5 Summary and open items

The controller lands successfully in the large majority of runs. In nominal conditions it meets the touch-down speed and chassis tilt requirements, it acquires the marker quickly, and it handled the tested perception edge cases (late acquisition, temporary marker loss, permanent low-altitude blackout) in most runs. A small number of runs (1 of 20 in the nominal set, and 1 of 6 in each stress scenario) did not end in a successful landing. Owing to limited time, these misses were not root-caused, and the stress scenarios contain only 6 runs each, which is too few for tight statistical claims. Root-causing these cases, in particular the missed-rover landing and the contact speed under permanent blackout, and repeating the campaign with more runs, is the first item of future work.

11. Limitations and future work

- Yaw is uncontrolled because the simulator applies only vertical point forces; on a real vehicle a yaw-torque channel and marker-yaw alignment (already computed as a diagnostic) would be added.
- Arena constants (marker plane and contact heights, camera pose, inertia) are hard-coded; they should be read from a configuration for other platforms.
- Body rates use a first-order finite difference of attitude; a gyro or a small filter would reduce noise in a real system.
- The Kalman filter uses a constant-acceleration model; an interacting-multiple-model filter could handle abrupt rover manoeuvres better.
- The cut-off height trades contact-detection robustness against impact speed (Sec. 7.2); a contact-force or velocity-drop detector would allow cutting at the pad.
- Validation coverage: the stress scenarios use 6 runs each and the remaining failed runs were not root-caused (Sec. 10.5); a larger randomised campaign is the natural next step.

Appendix A: Code map

Report section	controller.py element
3.1–3.3 Vision	`_detect_corners, _pixels_to_plane, _vision`
4.1 Attitude / rates	`quat_to_rot, rot_to_euler, _body_rates`
4.2 Kalman filter	`RoverKF.predict / update / reset`
5.2–5.3 Outer loops	`calculate_forces: horizontal and vertical reference, P→PI blocks`
5.4 Thrust vector	`calculate_forces: a_vec, T, pitch_des, roll_des`
5.5–5.6 Inner loop, mixer	`calculate_forces: rate_cmd, alpha, d vector, base shift`
6 State machine	`SEARCH / TRACK / DESCEND / BLIND_DESCEND / LANDED, _spiral_ref`
7 Descent	`_descent_profile, landing-cone gate s, Z_CUT latch`

Dependencies: Python 3, NumPy, OpenCV with cv2.aruco (opencv-contrib-python, 4.7 or newer for the ArucoDetector API), PyBullet for the arena. **Tested with:** Python 3.14.7, NumPy 2.5.3, OpenCV 5.0.0 (cv2.aruco OK), PyBullet (getAPIVersion = 202010061); platform macOS-26.6.2-arm64-arm-64bit-Mach-O.